

Progetto: EventHub

1. Panoramica del Sistema

EventHub è una piattaforma backend per la gestione e la prenotazione di eventi (concerti, conferenze, festival). L'applicazione permette agli **Organizer** di creare eventi inserendo dettagli su location (Sedi), categorie (Tag) e relatori (Speaker).

Gli utenti con ruolo **User** possono consultare il catalogo degli eventi con paginazione e ordinamento cronologico, prenotare biglietti (Standard o VIP) e lasciare un feedback (voto e recensione) solo dopo la conclusione dell'evento. Gli **Admin** monitorano l'intera piattaforma, gestiscono l'anagrafica globale e possono cambiare i ruoli degli utenti.

2. Entità del Dominio e Diagramma ER

Il modello dati è composto da 7 entità principali collegate dalle seguenti relazioni:

- **User** ↔ **Profile**: Relazione **1:1** bi-direzionale (un utente ha un solo profilo anagrafico).
- **User** ↔ **Role**: Relazione **N:M** (un utente può avere più ruoli, gestita tramite tabella di join **user_roles**).
- **Venue** → **Event**: Relazione **1:N** (una sede ospita molti eventi, un evento si tiene in una sola sede).
- **User (Organizer)** → **Event**: Relazione **1:N** (un organizzatore crea molti eventi).
- **Event** ↔ **Tag**: Relazione **N:M** (un evento ha più tag, un tag è associato a più eventi).
- **Event** ↔ **Speaker**: Relazione **N:M** (più relatori per evento, uno speaker può partecipare a più eventi).
- **Ticket**: *Join Entity* con relazione **N:1** verso **User** e **N:1** verso **Event** (contiene attributi extra come prezzo, tipo e stato).
- **Feedback**: Relazione **N:1** verso **User** (autore) e **N:1** verso **Event** (evento recensito).

erDiagram:

```
USER {  
  
    int id PK
```

2

string username

string email

string password

}

PROFILE {

int id PK

int user_id FK

string first_name

string last_name

}

ROLE {

int id PK

string name

}

VENUE {

int id PK

string name

string address

3

int capacity

}

EVENT {

int id PK

string title

date date

int venue_id FK

int organizer_id FK

}

TAG {

int id PK

string name

}

SPEAKER {

int id PK

string name

}

TICKET {

4

int id PK

int user_id FK

int event_id FK

string type

double price

string status

}

FEEDBACK {

int id PK

int user_id FK

int event_id FK

int rating

string comment

}

USER ||--|| PROFILE : "1:1"

USER }|..|{ ROLE : "N:M"

USER ||--o{ EVENT : "1:N"

USER ||--o{ TICKET : "1:N"

USER ||--o{ FEEDBACK : "1:N"

VENUE ||--o{ EVENT : "1:N"

EVENT ||--o{ TICKET : "1:N"

EVENT ||--o{ FEEDBACK : "1:N"

EVENT }|..|{ TAG : "N:M"

EVENT }|..|{ SPEAKER : "N:M"

3. Endpoint REST dell'Applicazione

Mappatura degli endpoint principali implementati nei controller con i relativi ruoli e permessi associati in [SecurityConfig](#):

Risorsa	Metodo	URL	Descrizione	Ruolo Accesso
Auth	POST	/api/auth/signup	Registrazione pubblica di un nuovo account utente	Pubblico (Anonimo)
User	GET	/api/users	Recupera la lista di tutti gli utenti	ADMIN

	GET	/api/users/{id}	Trova un utente specifico per ID	ADMIN
	PUT	/api/users/{id}	Aggiorna i dati di un utente	USER (proprietario), ADMIN
	DELETE	/api/users/{id}	Elimina un utente dal sistema	ADMIN
	PUT	/api/users/{id}/role	Modifica il ruolo di un utente	ADMIN
Profile	GET	/api/profiles/me	Recupera il profilo dell'utente loggato	USER, ORGANIZER, ADMIN
	GET	/api/profiles	Recupera tutti i profili	ADMIN
	GET	/api/profiles/{id}	Trova un profilo specifico per ID	ADMIN

	POST	/api/profiles	Crea una scheda profilo	USER, ORGANIZER, ADMIN
	PUT	/api/profiles/{id}	Aggiorna un profilo esistente	USER (proprietario), ADMIN
	DELETE	/api/profiles/{id}	Elimina un profilo	ADMIN
Event	GET	/api/events	Recupera gli eventi (Paginati e Ordinati per data)	Pubblico / Tutti
	GET	/api/events/{id}	Trova un evento specifico per ID	Pubblico / Tutti
	POST	/api/events	Crea un nuovo evento	ORGANIZER
	PUT	/api/events/{id}	Aggiorna un evento esistente	ORGANIZER
	DELETE	/api/events/{id}	Elimina un evento	ADMIN

	GET	/api/events/{id}/rating	Calcola la media voto matematica di un evento	Pubblico / Tutti
	GET	/api/events/date	Cerca eventi filtrando per data	Pubblico / Tutti
	GET	/api/events/venue	Cerca eventi filtrando per nome della sede	Pubblico / Tutti
Ticket	POST	/api/events/{id}/book	Prenota un biglietto (Standard/VIP) per un evento	Autenticato (Qualsiasi ruolo)
	DELETE	/api/tickets/{id}	Cancella/Annulla una prenotazione ticket	USER (proprietario), ADMIN
Feedback	POST	/api/feedBacks	Rilascia recensione (solo post-evento + ticket)	USER
	GET	/api/feedBacks	Recupera tutti i feedback inseriti	ADMIN

Venue	GET	/api/venues	Recupera tutte le sedi/location	Pubblico / Tutti
	POST	/api/venues	Crea una nuova sede fisica	ADMIN
	PUT	/api/venues/{id}	Aggiorna una sede esistente	ADMIN
	DELETE	/api/venues/{id}	Elimina una sede esistente	ADMIN
Speaker	GET	/api/speakers	Recupera tutti gli speaker/relatori	Pubblico / Tutti
	POST	/api/speakers	Registra un nuovo speaker nel sistema	ADMIN
	PUT	/api/speakers/{id}	Aggiorna lo speaker esistente	ADMIN
	DELETE	/api/speakers/{id}	Elimina lo speaker dal sistema	ADMIN

4. Pianificazione dei Livelli del Back-End

L'applicazione segue un'architettura a strati rigorosa e disaccoppiata. Di seguito la struttura dei pacchetti e l'elenco delle classi effettive del progetto:

com.academy.eventhub.entity

Contiene le classi modello mappate sul database relazionale tramite annotazioni ORM (JPA).

- **Classi:** User, Profile, Role, Venue, Event, Tag, Speaker, Ticket, FeedBack.

com.academy.eventhub.repository

Interfacce che estendono `JpaRepository` per fornire i metodi di persistenza dati e query custom.

- **Classi:** UserRepository, ProfileRepository, RoleRepository, VenueRepository, EventRepository, TagRepository, SpeakerRepository, TicketRepository, FeedBackRepository.

com.academy.eventhub.dto

Oggetti di trasferimento dati usati per validare l'input del client (`@Valid`) e nascondere informazioni sensibili in uscita.

- **Classi:** UserRequestDTO, UserResponseDTO, ProfileDTO, EventDTO, TicketDTO, FeedBackDTO, SpeakerDTO, VenueDTO.

com.academy.eventhub.mapper

Componenti software dedicati alla conversione automatica tra le classi del livello Entity e i rispettivi oggetti DTO.

- **Classi:** UserMapper, EventMapper, FeedbackMapper, ProfileMapper.

com.academy.eventhub.service

Lo strato della logica di business. Applica i vincoli transazionali, esegue le convalide e i calcoli complessi (es. calcolo media rating).

- **Classi:** UserService, ProfileService, VenueService, EventService, SpeakerService, TicketService, FeedBackService.

com.academy.eventhub.controller

Controller REST che espongono le API dell'applicazione, gestiscono i codici di stato HTTP e integrano i metadati per la documentazione automatica OpenAPI/Swagger.

- **Classi:** `AuthController`, `UserController`, `ProfileController`, `VenueController`, `EventController`, `SpeakerController`, `TicketController`, `FeedBackController`.

com.academy.eventhub.exception

Gestore centralizzato delle eccezioni. Intercetta gli errori a runtime (es. entità non trovate o fallimenti di validazione) e restituisce un JSON strutturato al client.

- **Classi:** `ErrorResponse`, `ResourceNotFoundException`, `GlobalExceptionHandler`.

com.academy.eventhub.security

Contiene la configurazione di sicurezza centralizzata dell'applicazione basata su Spring Security. Definisce le restrizioni di accesso (tramite `httpBasic`), l'approccio stateless, la codifica delle password con BCrypt e le query JDBC per caricare dinamicamente utenti e ruoli.

- **Classi:** `SecurityConfig`.